

字符串练习

str 中 substr 出现的次数

str = "hellolloabcllollo"

substr: "llo" strlen("llo")

写函数测试，llo 在 "hellolloabcllollo" 出现了多少次。

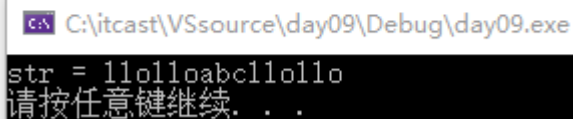
- strstr() 函数

```
1 #include <string.h>
2 char *strstr(const char *haystack, const char *needle);
3 char *strstr(const char *str, const char *substr);
4     参1: 原串
5     参2: 子串
6     返回值:
7         成功: 返回子串在原串中的位置 (地址值)
8         失败: NULL
```

- 测试strstr()

```
// 测试 strstr
int main(void)
{
    char* ret = strstr("hellolloabcllollo", "llo");
    printf("str = %s\n", ret);

    system("pause");
    return EXIT_SUCCESS;
}
```



count = 0;

strstr("hellollo", "llo") --> "llollo" --> count++

"llollo" += strlen("llo"); --> "llo"

strstr("llo", "llo") --> "llo" --> count++

"llo" += strlen("llo"); --> ""

strstr("", "llo") --> NULL

```

1 // 封装函数，统计 str字符串中，substr出现的次数
2 int substr_times(char* str, char* substr)
3 {
4     int count = 0; // 定义变量统计substr出现的次数。
5     char *p = strstr(str, substr); // ---> "lloabclloxyzllo";
6
7     // 循环的截取 str 串，判断剩余 str串中是否包含 substr
8     while (p != NULL) // while (p)
9     {
10         count++;
11         p += strlen(substr); // "lloabclloxyzllo";
12
13         p = strstr(p, substr);
14     }
15     return count;
16 }
17
18 // 在 str字符串中，找子串 substr出现的次数
19 int main(void)
20 {
21     char str[] = "hell9oqqqlloabclloxyzlmlo";
22     char substr[] = "llo";
23
24     int ret = substr_times(str, substr);
25     printf("%s 串中，%s 子串 出现 %d 次\n", str, substr, ret);
26
27     system("pause");
28     return EXIT_SUCCESS;
29 }

```

求字符串非空格元素个数

"ni chou sha ? chou ni za di ! zai chou yi ge shi shi" 统计这里，除空格外 字符的个数。

```

1 // 统计非空格数
2 int no_space_num(char* str)
3 {
4     int count = 0;
5     // 指针方式实现。
6     char* p = str;
7     while (*p)
8     {
9         if (*p != ' ')
10         {
11             count++;
12         }
13         p++;
14     }
15     return count;
16 }
17
18 int main(void)
19 {
20     char str[] = "hello ni hao ma world?";

```

```

21
22     int ret = no_space_num(str);
23
24     printf("ret = %d\n", ret);
25
26     system("pause");
27     return EXIT_SUCCESS;
28 }

```

字符串逆置

“hello” ---> “olleh”

“word” ---> “drow”

- 参考“数组逆置”实现。day05

```

1  // h e l l o
2  // 字符串的逆序
3  void str_inverse(char* str)
4  {
5      char* start = str;      // 记录首个元素的地址
6      char* end = str + strlen(str)-1;    // 记录最后一个元素的地址
7
8      // 循环交换字符串首尾元素
9      while (start < end)
10     {
11         char tmp = *start;    // 三杯水交换字符元素。
12         *start = *end;
13         *end = tmp;
14
15         start++;    // 首元素指针后移
16         end--;    // 尾元素指针前移
17     }
18 }
19
20 int main(void)
21 {
22     char str[] = "this is a test";
23
24     str_inverse(str);
25
26     printf("%s\n", str);
27
28     system("pause");
29     return EXIT_SUCCESS;
30 }

```

判断字符串是回文

abcba --- 是回文

amkilolikma --- 是回文。

abccba ---- 是回文。

abcdba --- 不是回文。

```
1 // 判断字符串是否是回文
2 int str_is_abcca(char* str)
3 {
4     char* start = str;
5     char* end = str + strlen(str) - 1;
6
7     while (start < end)
8     {
9         if (*start != *end)
10        {
11            return 0;        // 不是回文
12        }
13        start++;
14        end--;
15    }
16    return 1;
17 }
18
19 int main(void)
20 {
21     char str[] = "abcmnmcca";
22
23     int ret = str_is_abcca(str);
24     if (ret == 1)        // 是回文
25     {
26         printf("%s 是回文! \n", str);
27     }
28     else if (ret == 0)
29     {
30         printf("%s 不是回文! \n", str);
31     }
32
33     system("pause");
34     return EXIT_SUCCESS;
35 }
```

字符串处理函数

- 全部是标准C库函数。使用头文件 `#include <string.h>`

字符串拷贝

strcpy

```
1 char *strcpy(char *dest, const char *src);    // src:source    dest: dst
```

- 将 src 的内容，拷贝给 dest。返回 dest。dest空间要足够大。
 - strcpy 函数，不去检查 dest 是否足够大。—— 【不安全函数】

- 函数调用结束，返回值 和 dest 结果一致。

```
1 // 字符串拷贝
2 int main(void)
3 {
4     char str[] = "you will be die if you copy me!";
5
6     char dst[100] = { 0 };
7
8     char *p = strcpy(dst, str);
9
10    printf("dest = %s\n", dst);
11    printf("p = %s\n", p);
12
13    system("pause");
14    return EXIT_SUCCESS;
15 }
```

strcpy

```
1 char *strcpy(char *dest, const char *src, size_t n); // 安全
```

- 将 src 的内容，拷贝给 dest。只拷贝 n 个字节。dest 空间要足够大。通常 n 与 dest 的空间大小一致。

特性：

- n > src：只拷贝 src 大小。
- n < src：只拷贝 n 个字节。不会自动添加 \0

```
1 // 测试
2 int main(void)
3 {
4     char str[] = "hello world";
5
6     char dst[100] = { 0 };
7
8     char* p = strcpy(dst, str, sizeof(dst));
9
10    //for (size_t i = 0; i < 10; i++)
11    //{
12    //    printf("%c\n", p[i]);
13    //}
14
15    printf("%s\n", p);
16
17    system("pause");
18    return EXIT_SUCCESS;
19 }
```

字符串拼接

strcat

```
1 | char *strcat(char *dest, const char *src);
```

- 将 src 中内容，拼接到 dest 后。返回拼接成功的字符串。—— 需要保证 dest 空间足够大。
- 函数调用结束后，dest 和 返回值结果相同。

```
1 | int main(void)
2 | {
3 |     char str[] = "hello world";
4 |
5 |     char dst[100] = "haha hoho xixi";
6 |
7 |     char *p = strcat(dst, str);
8 |
9 |     printf("p = %s\n", p);
10 |    printf("dst = %s\n", dst);
11 |
12 |    system("pause");
13 |    return EXIT_SUCCESS;
14 | }
```

strncat

```
1 | char *strncat(char *dest, const char *src, size_t n);
```

- 将 src 中前 n 个字符，拼接到 dest 后。返回拼接成功的字符串。—— 需要保证 dest 空间足够大。
- 函数调用结束后，dest 和 返回值结果相同。

```
1 | // 字符串拼接 strncat
2 | int main(void)
3 | {
4 |     char str[] = "hello world";
5 |
6 |     char dst[100] = "haha hoho xixi";
7 |
8 |     char* p = strncat(dst, str, 7);
9 |
10 |    printf("p = %s\n", p);
11 |    printf("dst = %s\n", dst);
12 |
13 |    system("pause");
14 |    return EXIT_SUCCESS;
15 | }
```

字符串比较

- 字符比较可以使用 > < <= >= == != , 字符串 比较 不允许使用。

strcmp

```
1 | int strcmp(const char *s1, const char *s2);
```

- 比较 s1 和 s2 两个字符串，如果相等 返回 0；
- 如果不相等，进一步比 s1 和 s2 对应位上的 ASCII 码值。
 - s1 > s2 返回 1
 - s1 < s2 返回 -1

```
1 | int main(void)
2 | {
3 |     char s1[] = "helloz";
4 |
5 |     char s2[] = "helloaworld";
6 |
7 |     printf("ret = %d\n", strcmp(s1, s2));    // 不比较 ASCII 的 和
8 |
9 |     system("pause");
10 |    return EXIT_SUCCESS;
11 | }
```

strncmp

```
1 | int strncmp(const char *s1, const char *s2, size_t n);
```

- 比较 s1 和 s2 两个字符串的前 n 个字符，如果相等 返回 0；
- 如果不相等，进一步比 s1 和 s2 对应位上的 ASCII 码值。（不比较 ASCII 的和）

```
1 | char s1[] = "helloz";
2 | char s2[] = "helloaworld";
3 | printf("ret = %d\n", strncmp(s1, s2, 5));    --- 0
```

字符串格式化输入、输出

- s ---- string.

sprintf

```
1 | int sprintf(char *str, const char *format, ...);
2 | // ... 代表 这是一个参数可变的函数。
```

- 对应 printf 记忆。作用将 原来输出到屏幕的“格式化字符串”，写到 参1 的 str 中。

```

1 // printf("%d + %d = %d\n", 10, 24, 10+24);
2
3 char str[1024] = {0}; // 保证空间足够大
4 sprintf(str, "%d + %d = %d\n", 10, 24, 10 + 24); // 写到 str 中。不打印屏幕
5
6 puts(str);
7 printf("----%s", str);

```

sscanf

```

1 int sscanf(const char *str, const char *format, ...);

```

- 对应 scanf 记忆。作用 将 原来 从键盘获取到的“格式化字符串”，从 参1 的 str 中获取。

```

1 int a, b, c;
2
3 // scanf("%d+%d=%d", &a, &b, &c); // 从键盘 stdin 读取。
4
5 char str[] = "10+20=30"; // 提供给 sscanf 参1 使用。
6
7 int ret = sscanf(str, "%d+%d=%d", &a, &b, &c);
8
9 printf("a = %d\n", a);
10 printf("b = %d\n", b);
11 printf("c = %d\n", c);

```

字符串查找字符、子串

strchr

```

1 char *strchr(const char *s, int c);

```

- 在 字符串s 中，找 字符 c 出现的位置。返回 字符在字符串中的地址。

// 字符串中找字符出现位置 strchr

```

1 int main(void)
2 {
3     printf("%s\n", strchr("hellohehexixihoho", 'i'));
4
5     system("pause");
6     return EXIT_SUCCESS;
7 }

```

C:\itcast\VSsource\day09\Debug\day09.exe

ixihoho
请按任意键继续. . .

strrchr

- r: right

```

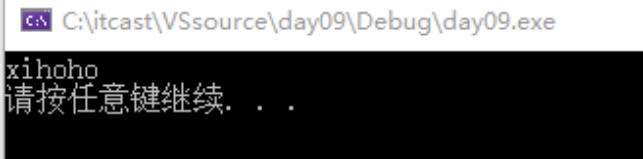
1 char *strrchr(const char *s, int c);

```


- 自右向左，在字符串s中，找字符c出现的位置。返回字符在字符串中的地址。

```
int main(void)
{
    //printf("%s\n", strchr("hellohehexixihoho", 'i'));
    printf("%s\n", strrchr("hellohehexixihoho", 'x'));

    system("pause");
    return EXIT_SUCCESS;
}
```



strstr

```
1 char *strstr(const char *str, const char *substr);
```

- 在字符串str中，找寻子串substr第一次出现的位置。返回地址。

字符串分割

strtok

```
1 char *strtok(char *str, const char *delim);
2     参1: 待拆分字符串
3     参2: 分割符组成的字符串。 strtok("www.baidu.com", "."); // 写成 '.' 错误!
```

- 按照（参2）既定的分割符，来拆分字符串。www.baidu.com 按“.”拆分。

```
1 // 测试1:
2 int main(void)
3 {
4     char str[] = "www.itcast.cn"; // ---》 "www\0itcast.cn";
5     char *p = strtok(str, "."); // strtok调用完成，会将 分割符用 \0 替换。
6
7     printf("p = %s\n", p);
8
9     // 调用一次 strtok 分割之后，再去打印原串 str
10    for (size_t i = 0; i < 13; i++)
11    {
12        //printf("%c\n", str[i]);
13        printf("%d\n", str[i]); // 打印每个字符的 ASCII码
14    }
15
16    system("pause");
17    return EXIT_SUCCESS;
18 }
```

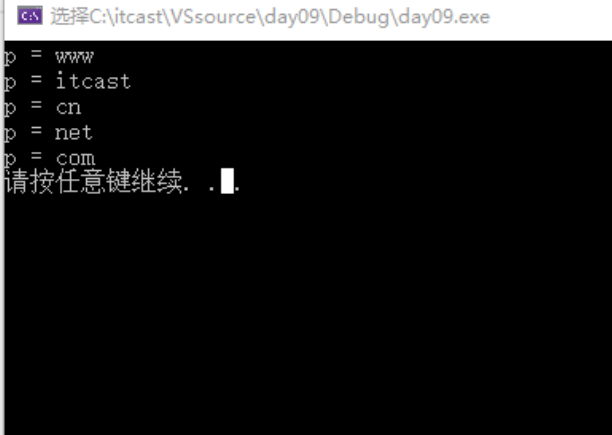
- 结论：
 - strtok() 函数，直接在原串上对字符串分割。不能分割字符串常量。char *str = "hello";
 - strtok() 函数调用结束，会将 分割符，替换成 '\0'
 - 第一次用 strtok 拆分，参1 传待拆分的原串。第 1+ 次拆分，参1 传 NULL。

```
// 字符串分割 strtok
int main(void)
{
    char str[] = "www.itcast.cn.net.com"; // ---》 "www\0itcast.cn";
    //char *str = "www.itcast.cn.com.net"; // 字符串常量。

    char *p = strtok(str, "."); // strtok调用完成, 会将 分割符用 \0 替换。

    printf("p = %s\n", p);

    // 循环按 "." 拆分 剩余 字符串。
    while (1)
    {
        p = strtok(NULL, ".");
        if (p == NULL)
        {
            break;
        }
        printf("p = %s\n", p);
    }
}
```



练习:

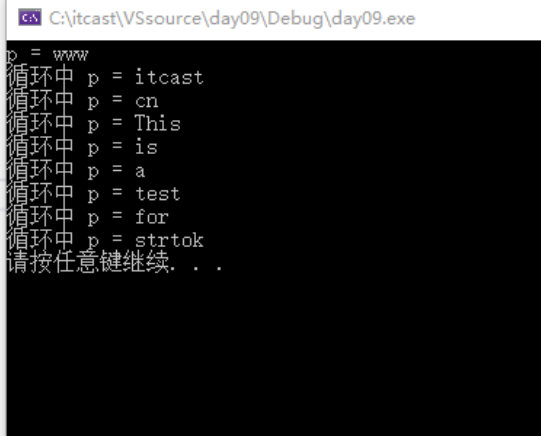
- 拆分字符串 `"www.itcast.cn$This is a test$for strtok"`, 按分割符 `". $"`...

```
//拆分 字符串 "www.itcast.cn$This is a test$for strtok", 按 分割符 ". $" ...
int main(void)
{
    char str[] = "www.itcast.cn$This is a test$for strtok";

    // 调用第一次
    char* p = strtok(str, ". $"); // 分割符有3个: '.' / '$' / ' ' 空格
    printf("p = %s\n", p);

    // 后续 N+1次调用。while循环, 参1传NULL
    while (1)
    {
        p = strtok(NULL, ". $");
        if (p == NULL)
        {
            break;
        }
        printf("循环中 p = %s\n", p);
    }

    system("pause");
    return EXIT_SUCCESS;
}
```



字符串转换

a: 代表字符串 string。

- 将字符串转整数、小数、长整数、长长整型。
- 使用这类函数进行转换时, 要求: 原串必须是可转换的字符串。
 - 错误使用: "abc123"、"xyac123"、"1245dke89" 不能正确转换。

atoi

```
1 #include <stdlib.h>
2 int atoi(const char *nptr);
3
4 double atof(const char *nptr);
5 long atol(const char *nptr);
6 long long atoll(const char *nptr);
```

示例:

```
1 char str1[] = "12abc3456";
2 int num = atoi(str1);
3 printf("num = %d\n", num);
4
5 char str2[] = "3.14";
6 double num2 = atof(str2);
7 printf("num2 = %lf\n", num2);
8
9 char str3[] = "34568490354";
10 long long num3 = atoll(str3);
11 printf("num3 = %lld\n", num3);
```

内存管理

局部变量

- 概念:
 - 定义在函数内部的变量。
- 作用域:
 - 从定义位置开始，到包裹该变量的第一个右大括号结束。（函数作用域、块作用域。）

```
1 void test(void)
2 {
3     {
4         int m = 10; // 局部变量。 --- 块作用域。出了 } 不能使用。
5     }
6     printf("---m = %d\n", m);
7
8     //int i = 0;
9     for (size_t i = 0; i < 10; i++)
10     {
11         printf("m = %d\n", m);
12     }
13     //printf("i = %d\n", i);
14 }
```

全局变量

- 概念:

- 定义在函数外部的变量。
- 作用域：
 - 从定义位置开始，默认到本文件内部。其他文件如果想使用，可以通过“声明”的方式，将作用域导出。

static 变量

static全局变量

- 定义语法：
 - 在全局变量定义之前，添加 static 关键字。如：static int a = 10;
- 作用域：
 - 被限制在本文件内部，不允许通过“声明”方式导出作用域。（java不同）

static局部变量

- 定义语法：
 - 在局部变量定义之前，添加 static 关键字。
- 作用域：
 - 从定义位置开始，到包裹该变量的第一个右大括号结束。
- 特性：
 - 静态局部变量，只定义一次。相当于，在全局位置定义。通常用来做计数器。

```
void test08(void)
{
    static int b = 10; // 静态局部变量
    printf("%d\n", b++);
}

int main(void)
{
    for (size_t i = 0; i < 10; i++)
    {
        test08(); // 调用 test08 10 次。
    }

    system("pause");
    return EXIT_SUCCESS;
}
```

C:\itcast\VSsour
10
11
12
13
14
15
16
17
18
19
请按任意键继续.

static 函数

- 全局函数：
 - 就是“函数”。
 - 定义语法：函数原型 + 函数体。
- static函数：

- 定义语法：static 函数原型 + 函数体。
- 特性：
 - static 函数，只能在 本文件内使用。其他文件即使声明也无法使用。

```

1 A.c 文件中有如下代码
2 // static 关键字能将 test09 函数限制在本文件内。  外部文件，不能访问
3 static void test09(void)
4 {
5     for (size_t i = 0; i < 5; i++)
6     {
7         printf("i = %d\n", i);
8     }
9 }
10 -----
11 B.c 文件中调用上述函数。由于 static ， B.c 不能使用 test09函数，会报错！
12 #define _CRT_SECURE_NO_WARNINGS
13 #include <stdio.h>
14 #include <string.h>
15 #include <stdlib.h>
16 #include <math.h>
17 #include <time.h>
18
19 // void test09(void); //声明函数
20 extern void test09(void); //声明函数
21
22 int main(void)
23 {
24     test09();
25
26     system("pause");
27     return EXIT_SUCCESS;
28 }
29

```

生命周期

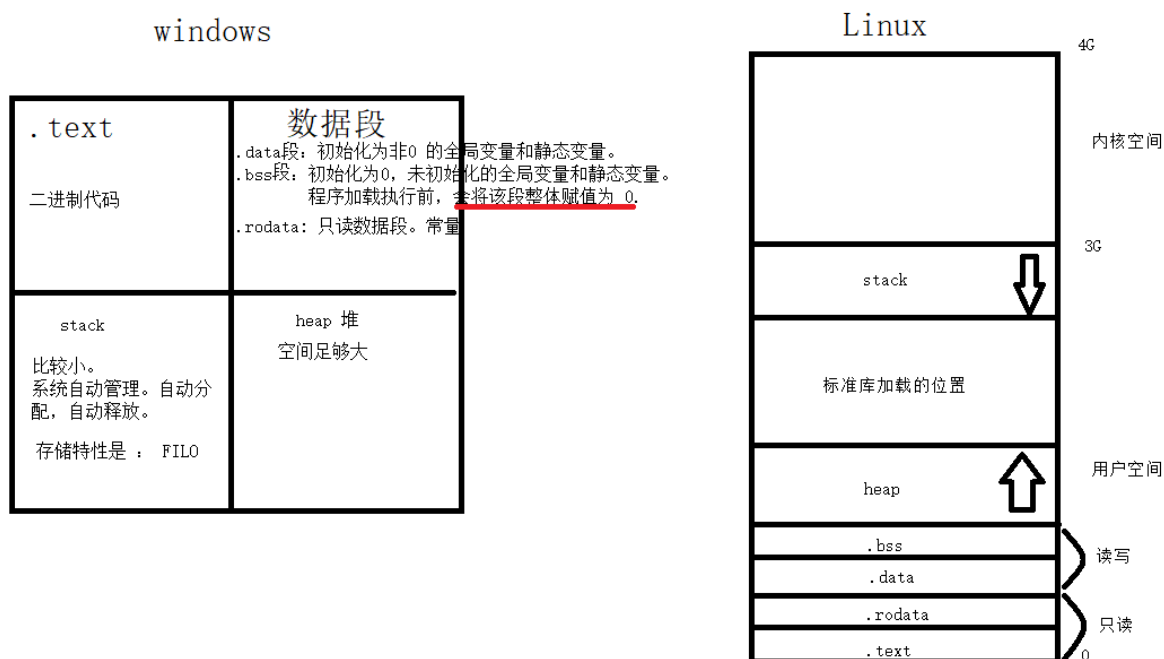
- 助记：
 - 生命周期：出生 ---- 死亡 80岁。生命周期：80年。
 - 作用域：班长：全班。村长：全村。县长：全县。
- 1. 局部变量：定一位置开始，函数调用结束（存储在栈stack上）。—— 函数被调用期间。
- 2. 全局变量：从程序启动开始（早于 main() 函数），程序终止结束。—— 程序执行期间。
- 3. static局部变量：从程序启动开始，程序终止结束（定义在全局位置）。—— 程序执行期间。
- 4. static全局变量：从程序启动开始，程序终止结束。—— 程序执行期间。
- 5. 全局函数：从程序启动开始，程序终止结束。—— 程序执行期间。
- 6. static函数：从程序启动开始，程序终止结束。—— 程序执行期间。

命名冲突

- 如果全局变量和局部变量命名冲突。采用就近原则。
- 强烈不推荐！

内存4区模型

1. 代码段：.text段。存储程序源码（二进形式）
2. 数据段：只读数据段 .rodata。初始化数据段 .data。未初始化数据段 .bss。
 - 3. stack：栈。在其之上开辟栈帧。（较小： windows: 1M--10M, Linux: 8M---16M）
 - 存储特性：后进先出 FILO (LIFO)
 - 4. heap：堆。给用户自定义提供空间。（较大：约1.3G+-）



Heap堆空间

开辟/释放 heap 空间

- 在 heap 上开辟空间

```
1 #include <stdlib.h>
2
3 void *malloc(size_t size);    // 向 系统申请内存空间，在 heap 上。单位：字节。
4     参数：申请空间的大小。
5     返回值：
6         成功：heap上内存空间的首地址。
7         失败：NULL
8     // 申请成功的内存，通常拿来当成“数组”使用
```

- 释放 heap 申请的空间

```
1 void free(void *ptr);
2     参：就是 malloc 函数的返回值。
```

- 示例：

```
1 int main(void)
2 {
3     // int arr[10];
4     // 申请能存储 10 个 int 数的空间。 40字节。
```

```

5      int *p = (int *)malloc(sizeof(int)*10);    // 强转的目的，方便阅读代码。
6      if (p == NULL)
7      {
8          printf("malloc error\n");
9          return -1; // 退出程序。 -1，非正常结束。
10     }
11
12     // 写 - 数据到 malloc 申请的空间
13     for (size_t i = 0; i < 10; i++)
14     {
15         p[i] = i + 10; // 存 数据到 malloc 申请的空间中。
16     }
17
18     // 从malloc申请的空间中，读数据。
19     for (size_t i = 0; i < 10; i++)
20     {
21         printf("%d ", *(p+i));
22     }
23     printf("\n");
24
25     // 释放malloc申请的空间。
26     free(p);
27
28     system("pause");
29     return EXIT_SUCCESS;
30 }

```

使用 heap空间注意事项

1. 申请的heap堆内存空间连续。当成“数组”使用。
2. free 后的空间，不会立即失效。通常将 free后的地址，置为 NULL。
3. free 地址必须是mallo函数返回的地址。否则，报错！
4. 如果malloc 后的地址一定会变化，通常使用临时变量 tmp 保存。

二级指针对应的 heap 空间

内存操作函数

memset

```

1 | #include <string.h>
2 | void *memset(void *s, int c, size_t n);

```

memcpy

```

1 | void *memcpy(void *dest, const void *src, size_t n);

```

memmove

```
1 | void *memmove(void *dest, const void *src, size_t n);
```

memcmp

```
1 | int memcmp(const void *s1, const void *s2, size_t n);
```

内存常见问题
